

```
#include <iostream>
```

C++ 개요와 개발 환경

C++ 프로그래밍 1장 · 첫 프로그램을 만들기까지

```
Hello, World!
```

이 장의 학습 목표



01

C++가 어떤 언어이고 어디에 쓰이는지 설명할 수 있다.



02

코드가 실행 파일이 되기까지의 과정을 이해한다.



03

컴파일러와 개발 도구(IDE)를 설치하고 사용할 수 있다.



04

간단한 C++ 프로그램을 작성하고 실행할 수 있다.

미리 알아 둘 용어

프로그램

컴퓨터가 할 일을 순서대로 적어 놓은 명령의 모음

프로그래밍 언어

사람이 컴퓨터에게 명령을 적기 위해 쓰는 약속된 언어

소스 코드

사람이 작성한 코드. C++에서는 보통 .cpp 파일에 저장

컴파일러

소스 코드를 기계어로 바꿔 주는 번역 프로그램

실행 파일

바로 실행할 수 있는 파일. 윈도우에서는 .exe

IDE

작성.번역.실행.오류 찾기를 한 화면에 모은 도구

1.1

C++의 역사와 특징

C++는 어떻게 태어났나

1979



개발 시작

비야네 스트롭스트롭이
벨 연구소에서 시작

1983



이름 C++

"C with Classes"에서
지금의 이름으로

1998



첫 국제 표준

C++98, ISO 표준으로
공식 관리 시작

C의 빠른 속도는 그대로 두고, 큰 프로그램을 체계적으로 만들 기능을 더
하고 싶었습니다.

C++

++는 C 언어에서
"값을 1 늘린다"는 뜻의 기호

→ "C보다 한 단계 발전한 언어"

C++는 계속 발전하는 언어

국제 표준화 기구(ISO)의 표준 = "C++ 문법은 이렇게 동작해야 한다"는 공식 규칙집



2011년부터는 약 3년마다 새 버전이 나옵니다. 이 교재는 C++20을 기준으로 설명합니다.

C++의 네 가지 특징



빠르다

하드웨어에 가깝게 동작해 적은 시간과 메모리로 일합니다.



여러 방식으로 짤 수 있다

절차지향·객체지향·일반화를 모두 지원하는 멀티 패러다임 언어입니다.



메모리를 직접 다룬다

성능을 끌어올리기 좋지만, 실수하면 오류가 생기기 쉽습니다.



C 언어와 잘 어울린다

C 코드 대부분을 그대로 쓸 수 있어 오랜 C 자산을 활용합니다.

멀티 패러다임: 프로그램을 짜는 세 가지 방식



절차지향

요리 레시피처럼 할 일을
순서대로 적는 방식

교재 2~5장



객체지향

자동차를 바퀴·엔진·핸들의
조합으로 보듯 객체 단위로 나눔

교재 6~10장



일반화 프로그래밍

어떤 자료형에도 쓸 수 있는
틀(템플릿)을 만드는 방식

교재 11~12장

C++는 어디에 쓰일까



게임

언리얼 엔진,
대작 PC·콘솔 게임



운영체제·시스템

윈도우·macOS의
일부 구성 요소, 드라이버



웹 브라우저

크롬, 파이어폭스의
핵심 엔진



임베디드

자동차 전자 장치,
가전제품, 로봇



금융·과학

초고속 주식 거래,
시뮬레이션

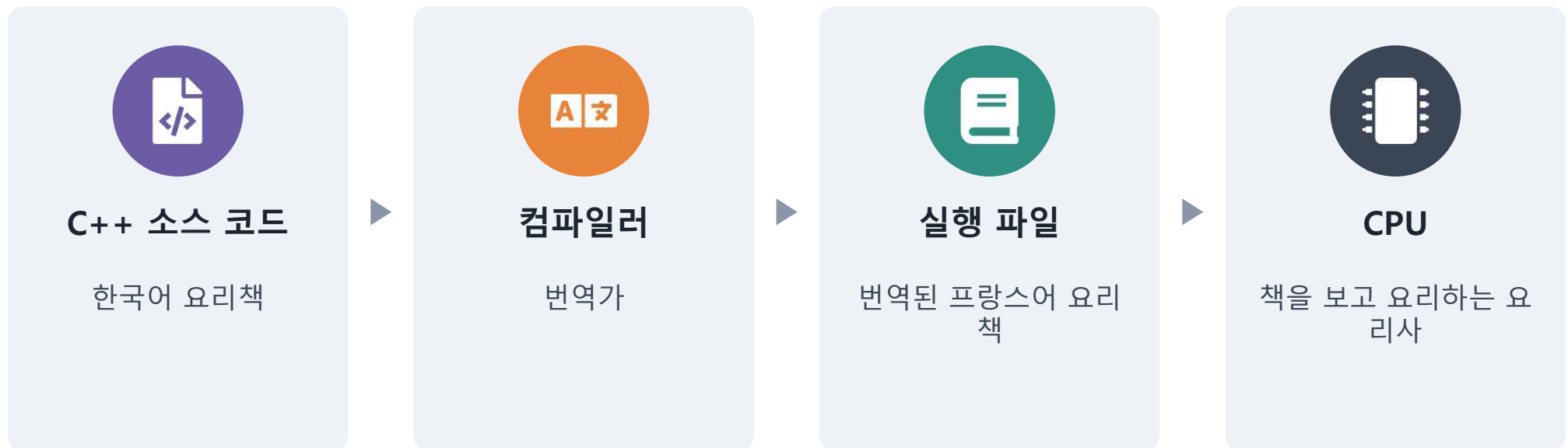
속도와 효율이 중요한 곳이라면 어디든 C++를 만날 수 있습니다. 처음엔 어렵지만, C++로 익힌 "컴퓨터가 실제로 어떻게 동작하는가"는 다른 언어를 배울 때도 큰 힘이 됩니다.

1.2

컴파일 과정

컴퓨터는 C++를 직접 알아듣지 못한다

CPU가 이해하는 언어는 0과 1로 된 기계어뿐 → 사람이 읽기 쉬운 C++로 쓰고, 컴파일러에게 번역을 맡깁니다. 이 번역을 컴파일(compile)이라고 합니다.



소스 코드가 실행 파일이 되기까지 4단계

1 전처리

#으로 시작하는 줄 처리
#include → 파일 내용 붙여넣기

hello.cpp → 펼쳐진 코드

2 컴파일

문법 검사 후 어셈블리어로 번역
문법 실수 → 컴파일 오류

→ 어셈블리어

3 어셈블

기계어로 바꿔 목적 파일 생성
아직 혼자 실행 못 하는 "부품"

→ hello.o / .obj

4 링크

목적 파일 + 표준 라이브러리 연결
부품을 못 찾으면 → 링크 오류

→ hello.exe

"빌드(build)" 버튼 하나를 누르면 이 네 단계가 차례로 일어납니다.

헤더 파일과 소스 파일



헤더 파일

.h .hpp

"이런 기능이 있다"는 선언(목록)

비유: 식당 메뉴판



소스 파일

.cpp

기능을 실제로 구현한 코드

비유: 주방의 실제 요리법

```
#include <iostream> // 메뉴판을 가져온다
```

1장에서는 표준 헤더 <iostream>만 사용하고, 헤더를 직접 만드는 방법은 4장에서 배웁니다.

오류의 세 가지 종류



컴파일 오류

2단계(컴파일)

세미콜론(;) 빠뜨림, 오타, 괄호 짝이 안 맞음



링크 오류

4단계(링크)

선언만 하고 구현을 안 함, main 함수가 없음



실행 오류

실행 중

0으로 나누기 등. 빌드는 성공했지만 실행 중 문제



팁

오류가 여러 개 나오면 맨 위의 첫 번째 오류부터 고치세요.

첫 오류 하나 때문에 뒤따라 생긴 오류가 많습니다.

오류 메시지는 컴파일러가 "여기를 고쳐 주세요"라고 알려 주는 친절한 안내입니다.

1.3

개발 환경 구축

필요한 도구: 컴파일러 + 편집기(IDE)

목수에게 톱과 작업대가 필요하듯, 코드를 번역할 컴파일러와 코드를 편하게 쓸 편집기 또는 IDE가 필요합니다.

MSVC

만든 곳 마이크로소프트
환경 윈도우

Visual Studio에 포함되어 함께 설치됨

GCC (g++)

만든 곳 GNU 프로젝트
환경 리눅스, 윈도우(MinGW)

무료, 오픈 소스. 가장 널리 쓰임

Clang

만든 곳 LLVM 프로젝트
환경 macOS, 리눅스

오류 메시지가 친절한 편. macOS 기본

세 컴파일러 모두 C++ 표준을 따르므로, 표준 문법으로 쓴 코드는 어느 컴파일러에서나 똑같이 동작합니다.

대표적인 개발 도구

도구	장점	추천 대상
Visual Studio Community	설치 한 번으로 컴파일러까지 준비. 디버거가 강력함	윈도우 사용 초보자 (교재 기본 환경)
Visual Studio Code	가볍고 다양한 언어 지원. 확장 기능으로 C++ 개발	컴파일러를 따로 설치할 수 있는 사용자
Xcode	macOS 공식 개발 도구, Clang 포함	macOS 사용자
CLion	똑똑한 코드 분석 기능	학생 무료 라이선스 이용자
온라인 컴파일러	설치 없이 웹 브라우저에서 바로 실행	설치 전 맛보기, 학교·도서관 PC

Visual Studio로 시작하기 (윈도우)

1 설치

공식 웹사이트에서 Visual Studio Community(무료)를 내려받아 실행

2 워크로드 선택

"C++를 사용한 데스크톱 개발"을 선택해 설치 (MSVC 포함)

3 프로젝트 만들기

새 프로젝트 만들기 → 빈 프로젝트

4 이름 정하기

프로젝트 이름(예: Hello)과 위치를 정하고 만들기

5 파일 추가

솔루션 탐색기 → 소스 파일 우클릭
→ 추가 → 새 항목 → C++ 파일
(main.cpp)

6 실행

코드 입력 후 Ctrl + F5 (디버그하지 않고 시작)

명령어로 직접 컴파일해 보기 (선택)

```
$ g++ -std=c++20 hello.cpp -o hello
# hello.cpp를 컴파일해 hello 실행 파일 생성
$ ./hello
# 실행
```

-std=c++20

C++20 표준 규칙으로 번역

-o hello

결과 실행 파일 이름을 hello로

clang++

Clang을 쓴다면 g++ 대신 입력



왜 명령어로 해 볼까?

macOS나 리눅스의 터미널에서는 컴파일러를 직접 실행할 수 있어, 컴파일 과정을 눈으로 확인할 수 있습니다. 설치가 막막하다면 온라인 컴파일러로 먼저 1.4절 예제를 실행해 보고, 막히면 오류 화면을 그대로 캡처해 질문하세요.

1.4

첫 번째 프로그램

Hello, World!

```
// 나의 첫 번째 C++ 프로그램
#include <iostream>

int main()
{
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

실행 결과

Hello, World!



화면에 글자가 나왔다면 첫
C++ 프로그램 완성!

프로그래밍을 배울 때 전통적으로 가장 먼저
만드는 프로그램입니다. 그대로 입력하고 실행
해 보세요.

한 줄씩 살펴보기

```
// ...
```

주석: 사람을 위한 메모. 컴파일러는 무시 (여러 줄은 /* */)

```
#include <iostream>
```

화면 출력.키보드 입력 기능을 가져옴 (전처리 단계)

```
int main()
```

프로그램의 출발점. int는 끝날 때 정수를 돌려준다는 뜻

```
{ }
```

main에 속하는 명령을 묶는 울타리. 짝이 맞아야 함

```
std::cout <<
```

화면으로 가는 통로에 오른쪽 내용을 흘려보냄

```
"Hello, World!"
```

큰따옴표 안의 문자열은 적힌 그대로 출력

```
std::endl
```

줄 바꿈. "Wn"을 써도 됨

```
;
```

문장의 끝. 가장 자주 빠뜨리는 기호!

```
return 0;
```

"문제없이 끝났다"는 신호. main에서는 생략 가능

std::와 네임스페이스



반 이름 붙이기

같은 반에 "김민준"이 둘이면 "1반 김민준", "2반 김민준"처럼 구분하듯,

std::cout = 표준(standard) 라이브러리 반의 cout

```
#include <iostream>
using namespace std;
// std:: 없이 쓰겠다는 선언
int main() { cout << "Hi"; }
```

짧은 연습 코드에서는 편리하지만, 큰 프로그램에서는 다른 이름과 충돌할 위험이 있습니다.
이 교재는 std::를 붙여 쓰는 방식을 기본으로 합니다.

좋은 습관, 그리고 일부러 오류 내 보기

좋은 코드 작성 습관

들여쓰기

중괄호 안은 스페이스 4칸(또는 탭 1번) 안쪽으로

대소문자 구분

Main, COUT은 오류. C++는 대소문자를 다르게 봄

주석 달기

"무엇을"보다 "왜" 그렇게 했는지를 적기



직접 해 보기: 일부러 틀려 보기

- 1 세미콜론(;)을 지운다
→ 컴파일 오류
- 2 `#include <iostream>`을 지운다
→ 컴파일 오류
- 3 `main`을 `Main`으로 바꾼다
→ 링크 오류 (1.2절 표와 비교)

정리

핵심 정리와 연습문제

핵심 정리



C++는 1979년 C를 바탕으로 개발된 빠르고 다재다능한 언어



표준은 2011년부터 약 3년마다 갱신, 교재는 C++20 기준



전처리 → 컴파일 → 어셈블 → 링크를 거쳐 실행 파일 완성



컴파일러(MSVC·GCC·Clang)와 편집기·IDE가 필요



모든 프로그램은 main에서 시작, std::cout으로 출력



문장 끝 세미콜론(;), 대소문자 구분에 주의

확인 문제와 프로그래밍 과제

확인 문제

1. C++라는 이름에서 ++가 뜻하는 것은?
 2. 소스 코드가 실행 파일이 되는 4단계를 순서대로
 3. #include <iostream>은 어느 단계에서 처리될까?
 4. 컴파일 오류와 링크 오류의 차이는?
 5. main 함수의 역할은?
 6. std::cout에서 std::는 무엇을 뜻할까?
-
7. 잘못된 곳 두 군데를 찾아 고치세요.

```
std::cout << "안녕하세요" << std::endl  
std::Cout << "반갑습니다" << std::endl;
```

프로그래밍 과제

1. 자기소개: 이름, 학과, 좋아하는 음식을 각각 다른 줄에 출력
2. std::cout을 여러 번 써서 별(*) 삼각형 출력
3. (도전) std::endl 대신 "\n"으로 다시 작성하고 결과 비교

```
*  
**  
***  
****
```

```
int score = 90;
```

다음 장 미리 보기

2장 기본 문법: 값을 저장하는 변수, 값의 종류인 자료형, 계산하는 연산자. 키보드로 값을 입력 받아 계산 결과를 출력하는 프로그램을 만듭니다.